General Introduction: Recursion is a great technique for problem solving, even beyond cases where your teacher required you to use it. However, sometimes implementing a solution recursively can lead to problems if sub-problems are repeated. Let's examine computing the Fibonacci numbers. Recall that $F_0 = 0$, $F_1 = 1$ and $F_n = F_{n-1} + F_{n-2}$. What happens if you implement that recursively? Is there a better way?

Weighted Interval Scheduling

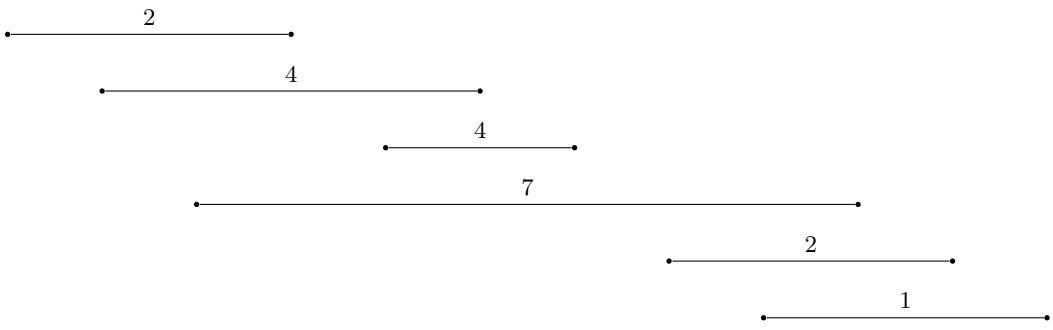
Warm-Up: we are given a set of n intervals, numbered $1 \dots n$, each of which has a start time s_i and a finish time f_i. For each interval, we want to compute a value $p[i]$, which is the interval j with the *latest* finish time f_j such that $f_j \leq s_i$; that is, the last-ending interval that finishes before interval i starts. If no intervals end before interval i begins, then $p[i] = 0$.

Give an $\mathcal{O}(n \log n)$ time algorithm that computes $p[i]$ for all intervals. You may assume that the intervals are already sorted by finish time.

The Big Problem: Fed up after the first midterm in CompSci 161, your friend has decided to change majors to one that grades based only on attendance. The only question is which classes your friend should take in Fall quarter. The classes all meet once a day, at different times and lengths, and are worth different amounts of credits. Your friend's goal is to maximize the amount of credits earned in that quarter without having to skip any classes (as this may interfere with passing those classes).

Problem Statement: More formally, we are given a set of n intervals, each of which has a start time s_i, a finish time f_i, and a value v_i. Our goal is to select a subset of the intervals such that no two selected intervals overlap and the total value of those taken is maximized.

Example Input: Please be aware that sample input will not always be provided in CompSci 161; one of the educational objectives is for you to be able to solve a problem in the abstract.



© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Let's solve this *recursively* (yay!). We will write a function `WIS(`i`)` that returns the optimal number of credits obtainable among intervals (classes) $1 \dots i$. We can then call `WIS(n)` to figure out the optimal number of credits obtainable among all intervals.

A key observation here is that your friend will either take class i or your friend won't take class i.

```
WIS(i)

    // Base Case:



   // If my friend doesn't take class i:

   value_if_not_taken =


   // If my friend takes class i:

   value_if_taken =


   //return something:
```

Should we implement it that way? We now have a recursive solution. Think back to the Fibonacci example earlier in lecture. What will happen if we implement this program this way?

We can now move to have a solution that uses no recursive *function calls*. Note that our solution is still conceptually recursive. The iterative solution will also allow us to output which courses to take, not simply the optimal value.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Homework Assignments (not for you)

Professor Shindler gives lots of homework assignments, each of which have an easy version and a hard version[1]. Each student is allowed, for each homework, to submit either their answer to the easy version (and get $e_i > 0$ points) or the hard version (and get $h_i > 0$ points, which is also guaranteed to always be more than e_i) or to submit neither (and get 0 points for the assignment). Note that e_i might have different values for each i, as might h_i. The values for all n assignments are known at the start of the quarter.

The catch is that the hard version is, perhaps not surprisingly, more difficult than the easy version. In order for you to do the hard version, you must have not done the immediate previous assignment at all: neither the easy nor the hard version (and thus are more relaxed, de-stressed, etc). You are allowed to do the hard version of assignment one if you want. Your goal is to maximize the number of points you get from homework assignments over the course of the quarter. Give an efficient dynamic programming algorithm to determine the largest number of points possible for a given quarter's homework choices.

[1] This is not the actual policy.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Longest Common Subsequence

Reading: G/T §12.5. **Problem Statement**: A *subsequence* of a given sequence is just the given sequence with zero or more elements left out. Given two sequences X and Y, we say that a sequence Z is a *common subsequence* of X and Y if Z is a subsequence of both X and Y. Our goal is to find the maximum length common subsequence.

Examples:

X	Y	LCS
complete	continue	cote
exercise	determine	eerie
surface	character	race
toward	thousand	toad

As with the previous lecture, let's determine the general recursive solution first. Can you determine something tautological about the LCS of sequences X and Y? Determine the recursive and iterative algorithms for LCS.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Edit Distance

Reading: Erickson, §3.7

The **Edit distance** problem is as follows. We are given two strings (not necessarily of equal length). We want to convert the first string to the other by a sequence of insertions, deletions, and substitutions. The **cost** is the number of operations we perform.

For example, if we want to convert FOOD to MONEY, we could do this:
FOOD $\rightarrow$ MOOD $\rightarrow$ MOND $\rightarrow$ MONED $\rightarrow$ MONEY

One way to visualize this is by alignment:

F	O	O		D
M	O	N	E	Y

We **define** Edit(i, j) to be the **minimum cost** to convert $X[1 \ldots i]$ to $Y[1 \ldots j]$.

What happened in the last column?

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Subset Sum

Reading: Erickson, §3.8, G/T §12.6 (related)

Problem Statement: Given a set S of n positive integers, as well as a positive integer T, determine if there is a subset of S that sums to exactly T.

Example 1: $S = \{2, 3, 4\}$, $T = 6$, the answer is "yes"

Example 2: $S = \{2, 3, 5\}$, $T = 6$, the answer is "no"

As with all other dynamic programming algorithms, we are going to start with a recursive case and transform it from there. Remember, *dynamic programming is about smart recursion.*

- Find a recursive algorithm to determine if a subset of the first n values in the input adds up to T.

- Finish the process to make this a dynamic programming algorithm, including outputting the subset of the items for the case when the answer is "yes." For example, your output on example one (above) should be "yes, 2, 4" while your output for example two should be "no."

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

What is the running time of the dynamic programming algorithm you gave for Subset Sum above?

- Suppose we double the size of S, but leave T alone. Will your algorithm scale well?

- Suppose we double the number of bits available to represent T, doubling its size, but leave S alone. Will your algorithm scale well?

One of the key terms here is *pseudo-polynomial*.

Knapsack

Suppose a hiker is about to go on a trek through a rainforest carrying a single knapsack. She knows the maximum total weight W she can carry and has a set S of n potentially useful items to take with her, such as a folding chair, a tent, or a printed copy of the CompSci 161 textbook. Each item i has a weight w_i (a positive integer) and a benefit value b_i (positive, not necessarily integer). The goal is to select a subset of the items such that the total weight is at most W while maximizing the total benefit of the items. This is known as the **0-1 Knapsack Problem**.

Give a dynamic programming algorithm to compute the largest subset of items she can carry with her, subject to the total weight constraint.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

Offline Optimal Binary Search Trees

Reading: Erickson §3.9, G/T 12.1 (related)

In ICS 46, you saw unbalanced binary search trees. These had $\mathcal{O}(\log n)$ lookup time under some conditions, but $\mathcal{O}(n)$ lookup time in the worst case. You then saw balanced binary search trees: there are structures such as AVL, Red/Black, WAVL, and Crumple Trees that make the "promise" that any given lookup in the tree would take $\mathcal{O}(\log n)$ time. We can't reasonably expect a better worst case, and these are great data structures for the case when elements can be added to the tree arbitrarily and we don't know how often (or even if) we will look up any given element.

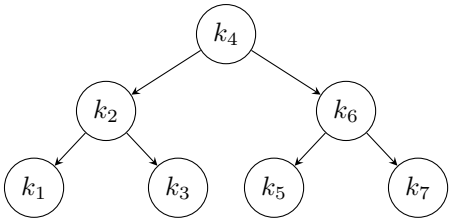
Suppose that before building a binary search tree, we knew exactly which elements were going to be in the tree. If we're likely to look up any given one with equal probability, or if we don't know the likelihood of looking up any given element, we can balance the tree by placing the median element at the root and recursively building trees in this fashion for the left- and right- subtrees.

But what if we also knew the probability that we'd look up any given element once the tree was built? This might not produce an optimal binary search tree in terms of the expected value of the lookup. Suppose we have n keys, $k_1 \dots k_n$, with probabilities $p_1 \dots p_n$ that we will look up the given elements; each probability p_i is positive, and the sum of these is 1.

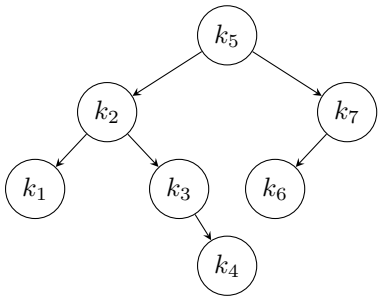
Here's an example with $n = 7$ keys:

i	1	2	3	4	5	6	7
p_i	.13	.21	.11	.01	.22	.08	.24

Here are two possible binary search trees with those keys:



This tree is balanced



This one is less balanced

What is the *expected* lookup cost (in terms of nodes examined) for each of these trees?

Problem Statement: We are given n probabilities, $p_1 \dots p_n$; p_i represents the probability of looking up the ith smallest key once the tree is built. Our goal is to build a binary search tree with the smallest expected lookup cost.
Note that our output must be a *binary search tree*; we cannot reorder the elements.

Check for understanding: we have computed d_i, the depth within the tree of each node. The root has $d_i = 1$, its children have $d_i = 2$, and so on. What is the expected lookup cost of this tree?

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

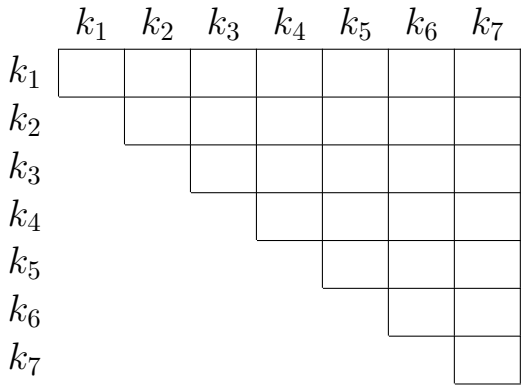
Creating the Dynamic Programming Algorithm

Let's compute Tree(i, j), which is going to be the *cost* of the optimal binary search tree consisting of keys i through j (inclusive). If this is called with $j < i$, we consider this a null tree and return 0 (treat this as a base case).

- Which key(s) can be the root of a binary search tree consisting of keys i through j?

- Suppose key r is the root. What is the cost of the search tree, rooted at r, consisting of keys i through j? You may assume that the left- and right- subtrees of r are constructed optimally.

Let's use that information to create a dynamic programming algorithm. When we're done, we will use that information to construct the tree itself.



The dynamic programming table before any are filled in. Any spaces that will remain unused are not pictured. I recommend filling in a few by hand (use a calculator as appropriate) to see the order in which they are filled in, and why that matters. See what happens, for example, if you tried to fill in the first row (in its entirety) first.

	k_1	k_2	k_3	k_4	k_5	k_6	k_7
k_1	0.13	0.47	0.69	0.72	1.28	1.52	2.12
k_2		0.21	0.43	0.46	1	1.17	1.73
k_3			0.11	0.13	0.47	0.63	1.19
k_4				0.01	0.24	0.4	0.95
k_5					0.22	0.38	0.92
k_6						0.08	0.4
k_7							0.24

The dynamic programming table after the program finishes with the sample input. Use this to check your work along the way if you take the previous advice.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

- Why can't we use the following to fill in the table of costs?

  for $i \leftarrow 1 \ldots n$ **do**
    for $j \leftarrow i \ldots n$ **do**
      Fill in Tree$[i, j]$

- In which order do we fill in table elements?

  for $i \leftarrow 1 \ldots n$ **do**
    Tree$[i, i-1] \leftarrow 0$
    Tree$[i, i] \leftarrow p_i$

Question 1. How do we get the tree itself from the table? Here is the table with the *costs* of the tree (this is the same table as on the previous digital page).

	k_1	k_2	k_3	k_4	k_5	k_6	k_7
k_1	0.13	0.47	0.69	0.72	1.28	1.52	2.12
k_2		0.21	0.43	0.46	1	1.17	1.73
k_3			0.11	0.13	0.47	0.63	1.19
k_4				0.01	0.24	0.4	0.95
k_5					0.22	0.38	0.92
k_6						0.08	0.4
k_7							0.24

Question 2. Write a function to, given the above table, construct the optimal binary search tree.

You should be able to reconstruct the tree from the table with the additions we make.

A good reinforcement activity would be to write a function that, given the full table, reconstructs the optimal tree.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.

StupidScript Programming

The StupidScript programming language includes a binary operator `@` that computes the *average* of two numeric arguments. For example, the StupidScript code `print(3 @ 6)` would print `4.5`, because $(3+6)/2 = 4.5$. Expressions like `4 @ 7 @ 3` that use the operator more than once yield different results when they are evaluated in different orders:

- $(4@7)@3 = 5.5@3 = 4.25$
- $4@(7@3) = 4@5 = 4.5$

Give a **dynamic programming** algorithm to compute, given a sequence of numbers separated by the @ sign, the *smallest* possible value an expression can take by adding parentheses. Your input is a vector $A[1 \ldots n]$, listing the numbers in the order. For example, if your input is $A = \{4, 7, 3\}$, your algorithm should return 4.25.

Recommended Problems

These questions will not be collected. Please do not submit your solutions for them. However, these are meant to help you to study and understand the course material better. You are encouraged to solve these as if they were normal homework problems, although if discussing these with classmates would help you learn it better, that is fine, as you are not submitting these for credit.

If you need help deciding which problems to do, consider trying R-12.7, R-12.8, C-12.1, C-12.9, C-12.16, A-12.1, A-12.2, A-12.3, A-12.4, A-12.6, A-12.10 in the textbook of Goodrich and Tamassia or Chapter 3, problems 1, 2, 3, 6, 9, 11, 12, 13, 16, 17, 18, 23, 32, 33, 35, 46, or 49 in the textbook of Erickson. For a big challenge, try question 28 in that chapter.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This handout may not be reposted without the express written permission of the professor teaching this course.